

# Use a Workflow When the Steps Are Known

A workflow follows declared steps and branches.



## CORE CONCEPT

# State, Nodes, and Edges Define a Graph

---

State is the typed record describing the current run. A node is a function that reads state and returns state updates. An edge determines which node runs next.

State is the typed record describing the current run.

A node is a function that reads state and returns state updates.

An edge determines which node runs next.

START represents graph input and END represents termination.

Store raw, reusable data in state rather than only formatted prose.

Every field should have an owner and meaning.

## RUN IT AND INSPECT THE RESULT

## A Node Should Perform One Inspectable Job

```
def retrieve_policy(state: State):  
    docs = retriever.invoke(state["question"])  
    return {"documents": docs}
```

**OUTPUT**

A state update containing documents while preserving other state fields.

**It does not generate an answer.**

## REASONING SEQUENCE

# Edges Make Routing Explicit

## Frame

**A fixed edge always routes to the same node.**

---

1

## Transform

**A conditional edge inspects state and chooses from declared destinations.**

---

2

## Validate

**Routing functions should return only allowed routes and should be tested independently.**

---

3

After retrieval: All three outcomes are explicit terminal or next states.

## CORE CONCEPT

# Tools Turn Model Choices into Proposed Actions

---

An agent model may propose: The application: Tool results are untrusted external data and may contain errors or injection text.

tool name;

structured arguments;

reason reflected in its current messages.

parses arguments;

authorizes the tool and scope;

applies time and resource limits;

## RUN IT AND INSPECT THE RESULT

# The Agent Loop Is Action, Observation, and Decision

```
def should_continue(state):  
    last = state["messages"][-1]  
    return "tools" if last.tool_calls else END
```

## OUTPUT

The graph routes to tools only when the latest model output proposes a call.

**Total tool calls 2, model decisions 3.**

## CORE CONCEPT

# Checkpoints Make State Durable

---

A checkpoint is a saved snapshot of graph state at an execution boundary. Checkpoints support: A thread ID identifies which persisted state to load. Access to checkpoints must be authorized because state may contain sensitive data.

recovery after failure;

multi-turn memory;

human approval pauses;

replay and debugging;

resuming long-running work.

## REASONING SEQUENCE

# Retries Must Be Node-Specific and Resume-Safe

## Frame

read-only search is  
often retryable;

---

1

## Transform

charging a card or  
sending an email may  
duplicate side effects;

---

2

## Validate

model calls may  
produce different  
outputs on retry;

---

3

## Explain

validation failures  
need correction, not  
blind retry.

---

4

Search times out: retry up to 3 attempts. Email send times out after the server may have accepted it: check an idempotency key or delivery status before retrying.



## RUN IT AND INSPECT THE RESULT

# Human Approval Is a State Transition

```
decision = interrupt({  
  "draft": state["draft"],  
  "recipients": state["recipients"],  
})
```

**OUTPUT**

The graph pauses with JSON-serializable review data until resumed.

Agent drafts an email and proposed recipients. The graph pauses. Reviewer removes one recipient and approves the edited draft. Only then may the send node execute.

## QUALITY GATE

# Stopping Conditions Bound Time, Cost, and Risk



maximum graph steps;

---



maximum attempts per node;

---



maximum tool calls;

---



wall-clock timeout;

---

## QUALITY GATE

# A Minimal LangGraph Is Explicit Python



**define typed state;**

---



**implement small node functions;**

---



**add nodes;**

---



**add fixed and conditional edges;**

---

## GUIDED LAB

## Guided Lab: Build a Bounded Research Workflow

01

---

**typed state**

02

---

**validate, route,  
retrieve, answer, and  
finish nodes**

03

---

**one  
deterministic  
route**

04

---

**one  
model-selected  
read-only tool**